

INSuRE Fall 2023 Project Proposal Malware Similarity

Sponsor: **MITRE**

Frank Posluszny
MITRE Technical Director
malwarefrank@mitre.org

Brian Ricks
UTD
Faculty
bwr031000@utdallas.edu

1. Venu Gopal Reddy Yendreddy
UTD
vxy210021@utdallas.edu

2. Neelima Reddy Kapa
UTD
nxk220027@utdallas.edu

3. Navya Teja Gajula
UTD
nxg220002@utdallas.edu

October 15th, 2023

Contents :

1. Project Summary

- 1.1 Top Level Project Description
- 1.2 Keywords
- 1.3 Terminology
- 1.4 Team Member Responsibilities
- 1.5 Estimated Project Budget
- 1.6 Summary of Key Deliverables

2. Executive Summary

3. Motivation

4. Literature Review

- 4.1 Malware Similarity Analysis: An adversary perspective - Frank Posluszny, March 2020
- 4.2 Software Forensics: Research Paper - Eugene H. Spafford and Stephen A. WeebeT
- 4.3 Automatic analysis of malware behavior using machine learning: Research Article - [Rieck, Konrad, Trinius, Philipp, Willems, Carsten, Holz, Thorsten](#)

5. Specific Aims

6. Research Plan/Approach

7. Anticipated Issues and Challenges

8. Broader Impact

9. Midterm Progress

- 9.1 Challenges

10. Investigator Biographies

Project Summary

1.1. Top Level Project Description

Malware similarity detection has been a longstanding challenge in the cybersecurity field, with various academic and operational efforts over the years. Unfortunately, there is still no clear consensus on the techniques or relevant malware features to effectively address this problem. To address this issue, we propose a fresh perspective by examining the fingerprints left behind by the malware development life cycle (mdlc) . Our research will investigate how to detect and measure these unique fingerprints, potentially reducing individual efforts and fostering cooperation among cybersecurity professionals to enhance digital security.

1.2. Keywords

Malware Similarity, Malware Analysis, Malware Development Life Cycle (MDLC), Mdlc fingerprints, Cyber adversaries, Adversary groups, Mitigation efforts, Malware variants, Opsec, Reverse Engineering, Obfuscation, Sandboxing.

1.3. Terminology

Malware : A software that is specifically designed to disrupt, damage, or gain unauthorized access to a computer system.

MDLC : An innovative malware analysis technique created at mitre intends to investigate the human component of malware production and how it might be exploited to learn more about malware samples and identify malware developers.

Artifact : An artifact is a concrete finding or pattern that can be drawn from a program and serves as a reminder of the decisions that were made during its development.\

Operation Security (OPSEC) : It is a method for identifying and stopping adversary progress

Reverse Engineering : The process of recovering information about an application from the earlier phases of the development lifecycle.

Malware Analysis : The process of understanding the behavior, original, and impact of a malware sample.

Static Analysis : The analysis of programs that involves examining the code without actually executing it.

Dynamic Analysis : The analysis of programs by monitoring the program when it is executed.

Obfuscation : Techniques used to delay or prevent analysis of a malware sample by making it harder to read or understand.

1.4. Team Member Responsibilities

Venu Gopal Reddy : Will serve as a team leader of the project and will be in charge of scheduling group meetings and reaching deadlines and will be in touch with our technical director of this project for further discussions . He has knowledge of technical aspects of the project and had an experience in c , c++ , python ,java which will be used in creating own scripts to develop this project .

Neelima Reddy Kapa : She is the assistant team lead for the project and in charge of organizing team internal meetings and collaboration times . She is well versed with machine learning libraries and python programming.

Navya Teja Gajula : She will be the researcher based on the project needs and she has experience in java , python . She is proficient in data analysis and will focus in incorporating he knowledge for this project.

1.5. Estimated Project Budget

This project is projected to take 533 hours to complete, which includes 8 hours per week for each researcher and 1 hour per week for the technical director. Currently, no monetary charges are necessary for the project because we are focusing on performing additional study to better understand the many different paths that our proposed statements could take. We do not believe there will be any software that is in the implementation phase that will cost money.

1.6. Summary of Key Deliverables

Key deliverables	Proposed due date
Project proposal	2/3
Phase one - complete research and decide which approaches to implement	3/3
Midterm presentation	3/10
Midterm report	3/12
Phase two - complete implementation of approaches	4/14
Phase three - evaluate and fine tune implementation of approaches	4/28
Final presentation	5/5
Final report	5/7

Executive Summary

MDLC as a way of summarizing all the various decisions and behaviors that result in a piece of malware being sent to an analyst. These actions and choices are made by those conceiving, creating, operating, collecting, and analyzing the malware. It is divided into 8 stages - requirements, design, code, build, configure, package, collection and analysis the inputs to, actions during, and outputs from the first six stages are affected by adversaries. They feed into the last two stages which are performed and affected by incident responders(defenders).each stage of the mldc leave fingerprints on the malware which can be helpful to determine similarity

The malware similarity can be identified in similarity of code , behavior , authorship and operational .code similarity can include code reuse , shared libraries algorithms, data structures. Behaviour similarity means taking similar actions or interacting the same way with an environment .authorship similarity is related to that if the same author were written the program, so that the code or behavior of the attack is similar . Operational similarity can be said as they were using same kind of operations but in a different way . The focused area of this project will be in understanding the humanistic elements of malware to analysis. To this stage we would do more research in the similar features and will map them to mdlc.

Motivation

In the modern digital landscape, malware has evolved into a sophisticated adversary that excels at evading detection. Malware creators employ a range of concealment techniques, such as encrypting their malicious code, employing polymorphism and metamorphism to constantly change their digital signature, and using advanced tactics like packing, where they hide the malicious payload inside seemingly harmless software. This complexity poses a significant challenge for cybersecurity analysts, making it challenging to identify and analyze similar malware samples. In light of the growing threat of advanced persistent threats (apts) and their ever-evolving attack methods, it is crucial to develop effective methods for detecting, tracing, and preventing malware attacks. Our motivation for this project stems from the need to extract structural information from malware samples to analyze their similarities and uncover hidden relationships, ultimately enhancing our ability to combat these evolving threats and bolster digital security.

Literature Review

4.1. Malware Similarity Analysis

Article Type : Research Paper

Authors : Frank Posluszny

The article presents the Malware Development Life Cycle (MDLC), mirroring the Software Development Life Cycle with eight phases. It discusses each stage and its relevance to malware analysis, emphasizing MDLC's utility in identifying similarities among malware instances, aiding practitioners in understanding adversaries' tactics. Frank Posluszny's research paper, 'Software Forensics', is a significant contribution to the field of digital forensics and software analysis. This research paper delves into the intricate process of tracing code back to its authors, a critical endeavor in identifying and attributing malicious activities in the digital realm. Posluszny's work provides valuable insights and methodologies for software forensics, offering techniques that aid in understanding the origins and authorship of code. This paper serves as an important reference for researchers and practitioners involved in cybersecurity, digital forensics, and software analysis, making it an essential addition to any comprehensive literature review in the domain.

4.2. Software Forensics

Article Type : Research Paper

Authors : Eugene H. Spafford and Stephen A. WeebeT

Anonymity is a key aspect that makes both computer crime and computer vandalism attractive activities. Whether the attack method involves viruses, worms, logic bombs, or unauthorized account access, it's generally very challenging to trace these actions back to specific individuals. An example is a hacker mentioned in a well-known case who was not worried about being caught even after detection, as he trusted that no one could trace his actions due to multiple network hops. Authors of viruses distribute them without fear of being identified.

To deter attackers, it's essential to reduce anonymity in computer use through measures like comprehensive session logging and improved network protocols with identification information. But there are limitations to the effectiveness of these methods, and no approach is entirely foolproof.

Handwriting analysis involves identifying specific features in writing, like the way letters are formed or the size of characters. Writer-specific features, which show small variations within an individual's writing and significant variations across different authors, are critical in identifying handwriting. It requires multiple samples to identify

these features accurately, as a person's handwriting changes over time and under different conditions.

Similarly, identifying authorship in computer code is possible by matching identifying characteristics. While code lacks physical features like handwriting, it offers room for variation and personalization, even when following detailed specifications. Programmers often have unique coding styles, and many texts recommend coding style elements. Statistical inference techniques, which have been used to verify authorship in prose, can also be applied to software, reducing subjectivity in the analysis.

Identifying the author of suspect code requires selecting an appropriate code sample and identifying relevant features for comparison. This may be challenging if the programmer attempted to conceal their authorship or if suitable sample code is unavailable. analyzing code characteristics may lead to suspect identification and insights into attack methods and vulnerabilities.

With sufficient background research and a large enough code sample, statistical methods applied to prose authorship determination may also be applicable to code. These methods, while not infallible, can potentially complement the analysis of stylistic features to provide clues about the authorship of code.

4.3. Automatic analysis of malware behavior using machine learning

Article type: Research Article

Authors: Rieck, Konrad, Trinius, Philipp, Willems, Carsten, Holz, Thorsten

Malicious software, commonly known as malware, represents a significant danger to computer system security. The sheer volume and diversity of malware variants have made traditional security defenses ineffective. As a result, millions of computers connected to the internet are infected with malware in the forms of computer viruses, internet worms, and Trojan horses. While malware often employs techniques like obfuscation and polymorphism to avoid detection when analyzed at the file level, dynamic analysis of malware binaries during runtime offers a means of understanding and defending against these threats.

In this article, there is introduction of a framework for the automated analysis of malware behavior using machine learning. This framework enables the automatic identification of new categories of malware with similar behavior (clustering) and the classification of unknown malware into these newly identified categories. Building on both clustering and classification, we propose an incremental approach to behavior-based analysis, capable of processing the behavior of thousands of malware binaries daily. This incremental analysis significantly reduces the runtime overhead compared to current analysis methods while maintaining accurate discovery and discrimination of new malware variants.

Specific Aims

1. Refinement of MDLC Later Stages Analysis:

Evaluate combinations and sequences of artifacts from the Build, Configure, and Package stages. Investigate unique indicators like specific command line arguments or packing methodologies. Assess the effectiveness of existing industry tools and prior work by the Spring 2023 UTD group.

2. In-depth Study of MDLC Earlier Phases:

Focus on Requirements and Design phases with emphasis on bytecode/source code analysis. Identify traces of original requirements and design choices in the code. Explore indicators of operational security (OPSEC) and relate them to requirements/design decisions.

3. Integration of Automated Similarity Analysis with MDLC:

Investigate how dynamic analysis and sandbox output can be effectively integrated with MDLC. Explore projects like Malheur for leveraging vectorized system calls in clustering and classification. Assess the potential of online/offline sandboxing tools for scalable malware similarity analysis.

4. Patterns in Obfuscation Techniques:

Identify patterns in obfuscation that may indicate specific authors or groups. Consider how unique obfuscation strategies can be used as indicators in malware categorization.

Research Plan / Approach

Our project will be executed in three distinct phases. The initial phase, commencing once we have completed the project proposal, will involve each team member conducting a comprehensive literature review of potential approaches to address the problem outlined in section 5. Throughout this phase, we will collaborate with both each other and our mentor to document our findings and any challenges associated with the proposed solutions. These discoveries will be documented and discussed during our regular weekly meetings. Our aim for the first phase is to enhance our comprehension of the problem domain and determine which solutions we intend to implement.

The second phase will revolve around the prototyping of the selected approach. Since the most effective solution will only become apparent after phase one, the specifics of this stage remain uncertain. However, the general approach will likely encompass a design phase in which the team outlines the steps involved in the solution, followed by an implementation phase in which we put the design plan into action. The approach employed in phase two may align with one of the methods outlined in section 5 or a novel approach discovered during the literature review.

By the conclusion of this phase, our objective is to possess a functional prototype or case study representing our chosen approach, enabling us to present our findings effectively.

In the third and final phase, we will conduct a comprehensive review of our initial approach and make necessary revisions. This phase is anticipated to be relatively brief, as the majority of the semester will be dedicated to research and solution implementation. During this phase, we intend to address any issues or glitches associated with our chosen problem-solving method. Additionally, we aim to identify potential challenges in our approach that can serve as valuable insights for future research in the field of malware similarity detection.

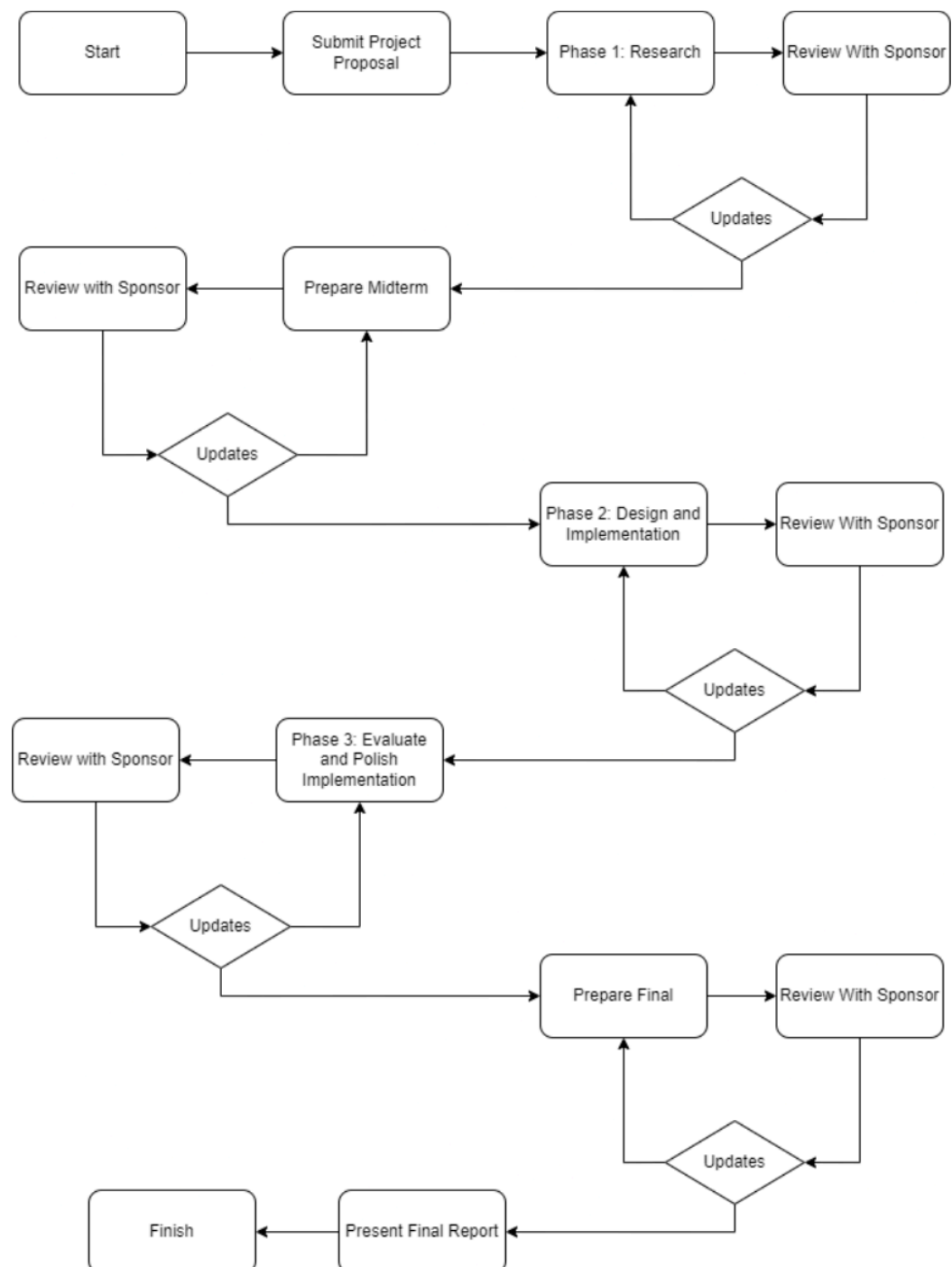


Figure 1: Research Plan Flowchart

Anticipated Issues and Challenges

This project is poised to encounter a series of challenges, necessitating proactive planning and resolution strategies. Firstly, the smaller team size could potentially lead to a heavier workload for individual members, demanding efficient resource allocation and time management. Additionally, the group's collective expertise in malware analysis and reverse engineering is varied, potentially resulting in differing levels of proficiency. This diversity of skills may require additional effort in knowledge sharing and collaborative learning to ensure that all team members are adequately equipped for the tasks at hand.

Furthermore, there may be a notable gap in domain-specific knowledge among team members, potentially impacting the depth of analysis required to extract meaningful insights from the available data. Addressing this challenge will involve targeted training and resource acquisition to bolster the team's collective knowledge base. Moreover, the compatibility of virtual environments across different operating systems, especially Windows and M1 Mac Machines, may pose logistical challenges. Ensuring seamless functionality and support for both Windows and M1 Mac environments will be crucial in maintaining an effective workflow throughout the project.

Finally, there is the potential for our initially proposed methods to face limitations in extracting artifacts from the Malware Development Life Cycle (MDLC) and establishing malware similarity. In the event that these methods do not yield the expected results, we will rely on our research and adapt our approach to a more effective alternative. These anticipated challenges underscore the importance of a dynamic and adaptable project strategy, with a focus on collaborative learning and resource optimization.

Broader Impact

Malware analysis entails the examination of a suspicious file through the utilization of various tools and procedures to comprehend its behavior and objectives. The team's primary focus lies in conducting similarity analysis among distinct malware instances. This endeavor holds the potential to assist security teams in swiftly identifying potential malicious entities before any harm befalls the organization. By gaining insights into the resemblances between different malware strains, the team's research ultimately aims to enhance the precision of categorizing and assessing malware attack incidents. Furthermore, the analysis of malware similarity also contributes to the automation of malware analysis. Over time, malware has not only grown in complexity but has also become more diverse and abundant. A more profound understanding of the similarities between malware specimens can enhance the speed and accuracy of automated malware detection processes.

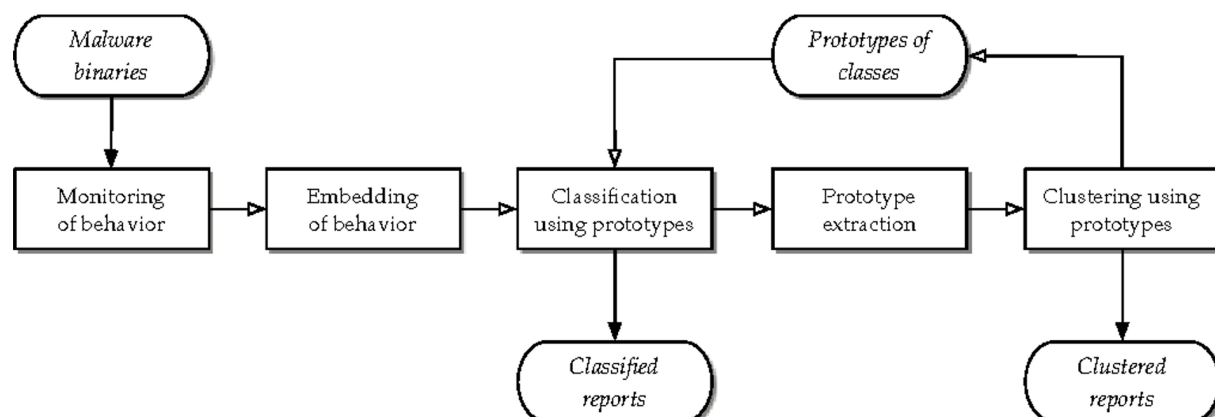
Midterm Progress

During the initial phase of our project, we conducted an extensive literature review to gain a comprehensive understanding of malware analysis methodologies. One key article, "Malware Similarity Analysis - An adversary perspective," provided valuable insights into the Malware Development Life Cycle (MDLC), drawing parallels to the Software Development Life Cycle. The MDLC, consisting of eight distinct phases, emerged as a pivotal framework for our analysis. We recognized the challenges posed by diverse malware types, data collection, integration into existing frameworks, and measurement of artifacts. To address these, our approach incorporates Malware Similarity Identification, Data Integration, Improved Defenses, and ongoing research efforts to refine our methodology.

Furthermore, the paper "Software Forensics – Can we track code to its authors?" highlighted the significance of combatting anonymity in cybercrimes. It underscored the importance of reducing anonymity through techniques such as detailed session logging and enhanced network protocols with user identification. Challenges including the effectiveness of countermeasures, authorship identification, and sample selection were identified. Our strategy involves implementing measures to Reduce Anonymity, improve Authorship Identification, and employ a rigorous Sample Selection process.

In parallel, we explored "Automatic analysis of malware behavior using machine learning," which introduced a novel approach to combating malware. The paper introduced a machine learning system to analyze malware behavior, enabling the detection and categorization of previously unknown malware types. We acknowledged the challenges associated with diverse malware behavior, real-time analysis, clustering, and classification, as well as the runtime overhead. To mitigate these challenges, we are employing Machine Learning techniques, conducting Incremental Analysis, adopting Behavior-Based Analysis, and refining methods for Accurate Discovery.

As a cohesive team of three, we are dedicated to leveraging the insights gained from these foundational articles to refine our approach and overcome the anticipated challenges in our pursuit of effective malware similarity analysis.



9.1 Challenges

Navigating with a smaller team size has necessitated a thoughtful allocation of resources and time management to ensure optimal progress. Additionally, our team members bring varying levels of domain-specific knowledge, which requires a concerted effort in knowledge sharing and collaborative learning. While we have access to a substantial volume of data, achieving deeper analysis for more meaningful insights is imperative. Furthermore, addressing the compatibility of virtual environments, especially the support for Windows and M1 Mac Machines, is crucial for seamless workflow integration. These challenges underscore the need for a dynamic and adaptable approach to maximize our project's success.

Investigator Biographies

Venu Gopal Reddy : I am currently in my second year of graduate studies, pursuing an MS in Computer Science at the University of Texas at Dallas. I did my undergrad in Electronics and Communication Engineering at NIT Manipur, India. I've got about 2.5 years of professional experience under my belt, having worked as a Software Engineer both in India and the USA. During my stint as a Java Spring Backend Developer, I got really comfortable with a bunch of development tools like Spring Tool Suite, DBeaver, Postman etc... Right now, I'm interning over at Copart as a Software Development Engineer. It's been a great opportunity to expand my skill set. I'm pretty strong in Java, and I've dabbled a bit in C/C++ and Python too.

Neelima Reddy Kapa : I am a second-year computer science major at UTD, with a previous undergraduate degree in computer science engineering from SRM University Chennai. I gained valuable experience during my undergraduate years, including a data science internship at MIT. This opportunity expanded my knowledge of machine learning libraries and strengthened my skills in Python. Additionally, I have a solid foundation in data structures, C, C++.

Navya Teja Gajula : I am passionate and driven computer science student with a journey that has taken me from the bustling cities of India to the heart of technology in Dallas, Texas. My quest for knowledge and fascination with technology led me to pursue a Bachelor of Technology degree in Electronics and Communication at the esteemed National Institute of Technology in Andhra Pradesh, India. Those four years were transformative, shaping my academic foundation and igniting my passion for computer science. Throughout my undergraduate years, I delved into a diverse array of courses, ranging from Data Structures and Discrete Structures to Machine Learning and Artificial Intelligence. These subjects served as the building blocks for my future endeavors, providing me with a solid understanding of the field's core principles.

One of the milestones in my academic journey was the transition to The University of Texas at Dallas, where I embarked on a Master of Science in Computer Science. This move marked a significant step towards honing my technical skills and exploring the limitless possibilities of technology in the heart of Dallas, Texas. In the realm of hands-on experience, I took on ambitious projects that pushed the boundaries of what I could achieve. One such project involved predicting stock market trends using advanced technologies like PySpark and recurrent neural

networks (RNN) with LSTM. This project not only tested my analytical abilities but also highlighted my capacity to leverage cutting-edge tools to tackle real-world challenges. Another captivating project involved text summarization using Latent Semantic Analysis (LSA), which allowed me to explore the fascinating world of natural language processing. By developing an LSA algorithm using Singular Value Decomposition (SVD) and TF-IDF, I unlocked the power to extract meaningful insights from extensive text documents. In the world of web development, I designed and implemented an E-commerce Website using React.js and Node.js, showcasing my proficiency in modern web technologies. Containerization with Docker and version control using Git were integral components of this project, underlining my commitment to delivering scalable and maintainable software solutions.

Beyond the classroom and technical projects, I have embraced opportunities to give back to the community. Serving as a Programming Instructor at CS Outreach, UT Dallas, allowed me to share my knowledge and passion for technology with incoming students. Collaborating on event sponsorships for our college's annual fest was an enriching experience that nurtured my leadership and teamwork skills.

References

1. Eugene H Spafford, Stephen A Weeber, "Software Forensics: Can we track code to its authors?", Purdue Technical Report CSD-TR 92-010, 19 February 1992.
2. Frank Poluszny, "MTR 200010 - Malware Similarity Analysis: An adversary perspective", The MITRE Corporation, March 2020.
3. Intezer. (2021, December 22). The role of malware analysis in cybersecurity. Intezer. <https://www.intezer.com/blog/malware-analysis/the-role-of-malware-analysis-in-cybersecurity/>
4. Cassiusm, P., Annelie, H., Olivier, Z., and Lamine, N. (2021, August 17-20). Accurate and Robust Malware Analysis through Similarity of External Calls Dependency Graphs (ECDG). IWCC'21, All-Digital Conference